

CS240: Lab 11 (30 pts)

TAs: Darshan

April 6, 2026

General Instructions

1. Welcome to the **graded Inlab** on Transformers. In this Inlab, you will work on three tasks in total. Each task is marked with stars (★) to indicate their difficulty and all the tasks are graded.
2. **IMPORTANT NOTE: DO NOT TRY TO GAME THE SYSTEM.** For each task, we have several checks and alternative tasks to ensure that you have solved the question as asked. Any deviations will attract harsh penalties.
3. In each task, you are only allowed to modify the **model.py** file. We will not entertain any regrading requests if the autograder fails. However, for your entertainment, you can tinker around the hyperparameters. For final grading, we will only use model.py file.
4. Use the `check-cs240` and `submit-cs240` scripts to submit. The final structure for the submission should look like this:

```
submission_{RollNumber}/
|-- task_1/
|   |-- model.py
|-- task_2a/
|   |-- model.py
|-- task_2b/
|   |-- model.py
|-- task_2c/
|   |-- model.py
|-- task_2d/
|   |-- model.py
|-- task_3/
|   |-- model.py
```

Task 1: Finding bugs (6 pts)

[★]

In this task, you are given the reference solution for Task 2 from the ungraded Outlab (the task where you were asked to implement Transformers for the Prefix-Sum problem). Unfortunately, the TA has messed up again. This time, he wrote the code in a state of post-lunch drowsiness, and introduced several bugs. Your mission is to identify and fix all of them. Just like last time, your solution code must not only run without errors but also demonstrate correct loss reduction during training. *[Hint: There are exactly six bugs in the code. That is, you will need to modify, insert, or delete exactly six lines to restore the correct code.]*

Expected Result

If you have resolved all the bugs, you should see the best test accuracy of $> 70\%$.

Task 2: Re-imagining Transformers (10 pts)

[★★]

In this task, you will modify the standard Scaled Dot-Product Attention mechanism to enforce specific structural constraints. Your objective is to adapt the core attention logic so that its attention pattern align with given geometric or logical priors. Since these constraints are easily verifiable visually, we have included helper code that saves plots of your attention scores directly to your working directory. Use the plots for debugging if needed. Also, unlike the previous task, here your implementation will be tested against several pre-seeded input-output pairs.

NOTE: The input sequences for this task will be of varying length N . Your implementation must be able to handle this. You can assume that all the examples within a batch will be of same length.

a. Right-to-Left (R2L) Attention (2 pts)

While standard Causal Attention (L2R) restricts a token to attending only to its predecessors, several research works have shown that Right-to-Left (R2L) attention (where a token at index i attends to all the tokens at indices $j \geq i$) can be equally effective for building context in non-autoregressive settings. While unsuitable for traditional language generation, R2L attention is a powerful tool for bidirectional encoders or "backward" language models. Your task is to implement this R2L attention.

b. Windowed Attention (2 pts)

In many specialized domains such as DNA Sequencing, a token typically requires only tokens in its close neighborhood to obtain all relevant information. In such scenarios, windowed attention serves as a computationally efficient alternative where we restrict the receptive field of every token to a localized span. That is, given a window size K ($K \geq 1$), each token is constrained to attending only to the K tokens immediately preceding and following its current position, including itself. Formally, a token at index i may attend to a token at index j if and only if $|i - j| \leq K$. Note that if $K \geq N$, this mechanism naturally reverts to standard global attention. Your task is to modify the attention scoring logic to enforce this windowing mechanism.

c. One-hot Attention (3 pts)

Like Windowed Attention, there are certain situations where we require attention to act as a discrete selector rather than an interpolator of information across the temporal dimension. In such scenarios, we use One-hot Attention, where a token is restricted to attending solely to the single most relevant token in the sequence. Specifically, the output vector for each position i must be the exact value vector $v_j \in \mathbb{R}^d$ corresponding to the token j that achieved the highest energy score. Your task is to modify the attention mechanism to bypass the standard weighted average and instead perform this "hard" selection.

d. Symmetric Attention (3 pts)

In graph-related tasks, relationships between positions are often inherently bidirectional. That is, if token i identifies token j as highly relevant, the reverse must also hold true. To model this behavior, you will implement Symmetric Attention, where the attention score assigned by token i to token j is strictly identical to the score assigned by token j to token i . Your objective is to modify the scoring logic to enforce this behaviour. Crucially, you must achieve this using vectorized tensor operations rather than manipulating learnable parameters, explicit loops, or direct element-wise assignments (e.g., don't write $A[i, j] = A[j, i]$ within loops). Such non-vectorized operations are incompatible with CUDA Graph capture and will result in significant performance bottlenecks in high-performance computing environments. *[Hint: Instead of trying to manipulate the attention matrix A , think of how you can use A to get a symmetric attention matrix.]*

Task 3: Block-Recurrent Transformers (14 pts)

[★★★]

In this task, you will implement a hybrid sequence-to-sequence architecture to bridge the fundamental gap between RNNs and Transformers. Recall from the class that, while Transformers excel at capturing long-range dependencies, their self-attention mechanism suffers from quadratic computational complexity ($O(N^2)$), making them prohibitively expensive for very long sequences. Conversely, RNNs process

data with linear complexity ($O(N)$) and possesses strong local inductive bias, yet they struggle with parallelization and "vanishing gradients" over very long sequences. To leverage the strengths of both paradigms, you will implement a new sequence-to-sequence architecture called Block-Recurrent Transformers. In this architecture, instead of performing global attention on every individual token, you will first partition the input into small, fixed-sized blocks. An RNN will then "summarize" each block by processing its tokens and producing a single summary token, which is simply the final hidden state of the RNN after the block is processed. These summary tokens act as compressed representations for the block, which are all then passed to a Transformer layer to "spread" global information across the entire sequence at a significantly reduced computational cost. In this way, you use the power of the RNN to distill local, sequential patterns into dense representations, and the power of the Transformer to efficiently propagate that information across distant parts of the sequence. The exact implementation details are as follows:

Step 1: Block Partitioning and Recurrent Compression

The input sequence $\mathbf{X} = \{x_1, x_2, \dots, x_N\}$, where $x_i \in \mathbb{R}^d$, must be partitioned into M non-overlapping "blocks" of fixed size K (where K is a hyperparameter and for simplicity, you can assume $N \% K = 0$), such that $M = N/K$. For each block $\mathbf{B}^i = \{x_{(i-1)K+1}, \dots, x_{iK}\}$, you will apply a shared unidirectional RNN unit with the initial hidden state (h_0^i) set to zero. This RNN iterates through the K tokens of the block, producing the outputs and the hidden states. From this, you will use the final hidden state h_K^i as the summary token s_i for the block. The exact formulation is as follows:

$$h_t^i = \sigma(Wh_{t-1}^i + Ux_{(i-1)K+t} + b) \quad \forall 1 \leq t \leq K \quad (1)$$

$$o_t^i = Vh_t^i + c \quad \forall 1 \leq t \leq K \quad (2)$$

$$s_i = h_K^i \quad (3)$$

where $W, U, V \in \mathbb{R}^{d \times d}$, $b, c \in \mathbb{R}^d$ are learnable weight parameters and σ is the activation function (typically tanh). Note that, by doing so, you reduce the effective sequence length for the Transformer from N to M , creating an information bottleneck that forces the RNN to distill the most salient features from the block.

Step 2: Global Contextualization

The resulting sequence of summary tokens $\mathbf{S} = \{s_1, s_2, \dots, s_M\}$ is then fed into a standard Transformer Encoder. You might want to add positional embeddings here. The output of this stage is a sequence of contextualized summary vectors $\mathbf{Z} = \{z_1, z_2, \dots, z_M\}$, where each z_i contains information from both its local block and the global context of the entire sequence.

Step 3: Feature Expansion and Context Injection

Finally, you must infuse the global context $\mathbf{Z}$ back into the original sequence. To do this, each contextualized summary vector z_i is broadcasted (repeated) K times to match the length of its corresponding original block. You will then merge this global context with the local RNN outputs o_t^i via element-wise addition. The final output of the model for a token at position t within block i is given by:

$$o_t^i = o_t^i + z_i \quad \forall 1 \leq t \leq K \quad (4)$$

This ensures that each output token embedding is informed by both the fine-grained local features captured by the RNN and the broad, long-range dependencies resolved by the Transformer.

Expected Result

If your implementation is correct, you should see a best test accuracy of $> 52\%$. Please note that model capacity and task parameters have been intentionally scaled down to ensure compatibility with lab hardware. While these constraints result in lower absolute accuracy than typical production settings, the architectural principles remain identical to high-performance real-world systems.

— THE END —